

Golden Rule -> chunk small enough to be specific, big enough to keep context, then overlap a little

1. Why Chunking Matters

Embedding models and LLM context windows both have limits, and stuffing a whole document into one vector blurs its meaning. Chunking splits documents into smaller, focused pieces so retrieval can find the exact passage relevant to a question instead of returning (or missing) an entire document.

2. Chunking Methods

Method	How It Splits	Good For
Fixed-size (CharacterTextSplitter)	Every N characters, blindly	Quick prototypes, very simple text
Recursive character splitting	Tries paragraph -> sentence -> word boundaries, in order	Best general-purpose default
Token-based	Splits by token count (e.g. tiktoken)	Matching embedding/LLM token limits precisely
Sentence / semantic splitting	Groups by sentence or meaning boundaries	Preserving full ideas; FAQ/Q&A-style docs
Markdown / HTML-aware	Splits along headers or tags	Structured docs: documentation sites, READMEs

3. Chunk Size & Overlap Guidance

- Typical starting point: 500–1000 characters (roughly 200–400 tokens), with 10–20% overlap.
- Smaller chunks -> more precise retrieval, but each chunk loses surrounding context.
- Larger chunks -> more context per chunk, but noisier retrieval and higher risk of exceeding limits.
- Overlap exists so a key sentence sitting right at a chunk boundary doesn't get cut in half and lost.

4. Code Example

```
$ from langchain.text_splitter import RecursiveCharacterTextSplitter

$ splitter = RecursiveCharacterTextSplitter(
$   chunk_size=800,
$   chunk_overlap=150,
$ )

$ chunks = splitter.split_documents(docs)
```

5. How To Choose

Document Type	Recommended Splitter
Structured docs (markdown/code)	Structure-aware splitter (headers/tags)
Long narrative or policy text	Recursive splitter, medium chunk size
Q&A / FAQ-style docs	Split by natural Q&A units, not fixed size

Golden Rule -> chunk small enough to be specific, big enough to keep context, then overlap a little

Document Type	Recommended Splitter
Strict token budgets	Token-based splitter

Pitfall

Splitting purely by character count can slice a sentence — or a table row — exactly in half, leaving both halves meaningless on their own once embedded.

Remember

There is no single “correct” chunk size. Always validate empirically: run sample questions through retrieval and check whether the returned chunks actually contain the answer.

INTERVIEW PREP

Q: Why not just embed the entire document as one chunk?

A: Embedding models compress meaning into a fixed-size vector — an entire long document blurs many topics into one vector, making similarity search imprecise. Chunking lets retrieval target the specific passage that answers a query.

Q: What is chunk overlap for?

A: It repeats a small amount of text between consecutive chunks so a sentence or idea split across a chunk boundary still appears fully intact in at least one chunk.

Q: When would recursive character splitting be preferred over fixed-size splitting?

A: When you want to preserve natural structure: it tries splitting on paragraph, then sentence, then word boundaries before falling back to raw characters, producing more coherent chunks than a blind fixed-size cut.

Q: How do you decide chunk size in practice?

A: Start from a reasonable default (e.g. 500–1000 characters), then empirically test retrieval quality on sample questions and adjust — too small loses context, too large dilutes relevance.

Q: What's a risk of chunk sizes that are too large?

A: Each chunk covers multiple topics, so its embedding becomes a blurred average, and irrelevant text ends up injected into the LLM's context alongside the relevant part.